

Project Proposal, Project Plan, and Risk Management
Deliverable I

Abstract

Team Falcons proposes Lost and Found for Campus, a MERN based web application that lets students, faculty, and staff post, search, and resolve lost and found items, extended with a matching engine, claim verification, messaging, a campus map view, and an admin dashboard to give the project sufficient depth for a four person team. This deliverable presents the project proposal, a high level timeline aligned to the course's five deliverable milestones, a risk management plan, team roles, and each member's contribution to this submission.

Team Falcons

Waleed Mirza, Venkata Sai Anirudh Naidu Mamidi, Sai Sathvika Potluri, Sai Asritha Thati

Deliverable I: Project Proposal, Project Plan, and Risk Management

Project Title: Lost and Found for Campus (Expanded Scope)

Group Name: Falcons

CSCE 5430, Software Engineering, Fall 2026

1. Project Title, Group Name, and Group Members

Project Title: Lost and Found for Campus (Expanded Scope)

Group Name: Falcons

Group Members:

- Waleed Mirza (ID: 1194980), Team Leader
- Venkata Sai Anirudh Naidu Mamidi (ID: 11908399)
- Sai Sathvika Potluri (ID: 11813771)
- Sai Asritha Thati (ID: 11909319)

2. Project Description

Team Falcons is developing Lost and Found for Campus, a web application that helps students, faculty, and staff report, search for, and recover lost items on campus. The core problem the system addresses is that lost items on a large campus are currently reported and tracked informally, through scattered social media posts, physical bulletin boards, or a single lost and found office that many students never think to check. This makes it slow and unreliable for someone who has lost an item to find out whether it has been turned in, and equally slow for someone who found an item to identify and notify its owner. Our application centralizes this process into a single searchable platform.

Primary stakeholders include students and staff who lose or find items day to day, campus security, and a campus lost and found office that would benefit from a centralized view of unresolved items. Because the workflow of posting an item, searching for a match, and resolving a claim is intuitive, the system is also easy for non technical evaluators, including the instructor and the teaching assistant, to understand and assess in a short demo.

Core Workflow: A user creates a listing for a lost or found item, including a photo, a location, a written description, and a category. Other users can search and filter listings by keyword, category, location, and date range to identify a possible match. Once an item has been returned or claimed, the original poster marks the listing as resolved so the board of active listings stays current and useful.

Beyond this core workflow, the system includes several additional modules that give the project meaningful depth and divide naturally into substantial, ownable pieces of work for a four person team. These modules are described below, and their placement within the overall system is shown in the architecture diagram that follows.

- Automated matching engine: compares new lost and found posts by category, location, date proximity, and description keywords, and proactively surfaces likely matches to both the finder and the person who lost the item, rather than requiring either side to browse manually.
- Notification service: sends in app and email alerts when a potential match is found, when a claim request is submitted, or when a listing is resolved, so users do not need to keep checking the site.

- Claim verification workflow: before a match is confirmed, the claimant submits identifying details about the item, and the original poster reviews and either approves or rejects the claim. This reduces the chance of a false or fraudulent claim succeeding.
- In app messaging: once a claim is approved, a moderated chat opens between the finder and the claimant so they can coordinate a handoff without exposing personal contact details such as phone numbers directly to each other.
- Interactive campus map view: listings are plotted on a campus map by building or zone, giving users a spatial way to browse lost and found items in addition to the standard list and search views.
- Admin and analytics dashboard: a dedicated role for campus security or the campus lost and found office to oversee unresolved items, monitor the overall resolution rate, and identify hotspot locations and common item categories over time.
- Role based access control: the system distinguishes between regular users, campus staff or moderators, and administrators, so that sensitive actions such as dashboard access are limited to appropriate roles.

Development Environment: The application follows a MERN style architecture. The front end is built with React (JavaScript, ES6 and later), and the back end is built with Node.js and the Express framework, exposing a REST API consumed by the front end. Data is stored in MongoDB, using the free tier of MongoDB Atlas for development, which fits the semi structured nature of listings, match records, and chat messages well. Nodemailer, or a comparable transactional email provider, handles match and claim notifications, and Leaflet paired with OpenStreetMap provides the campus map view without requiring a paid mapping API key during development. Development tools include Visual Studio Code, npm for package management, Postman for API testing, GitHub for version control, and Trello for Kanban based issue tracking. The application targets modern desktop and mobile web browsers and is intended to be deployable to a free tier hosting platform, such as Render or Vercel, for demonstration purposes.

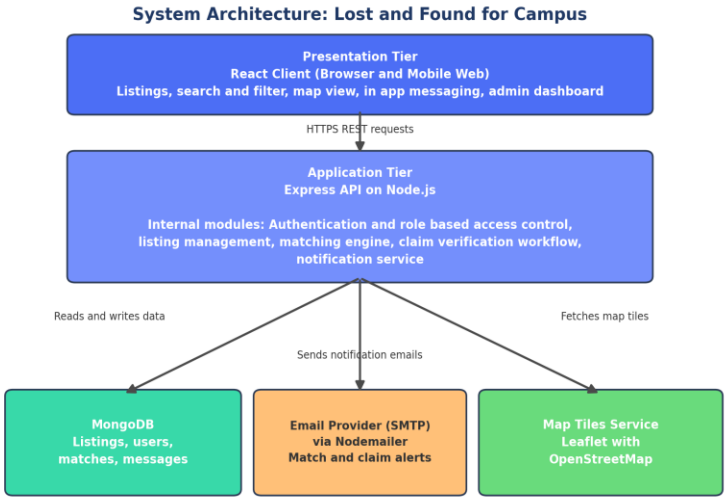


Figure 1. High level system architecture showing the presentation, application, and data and external service tiers.

As Figure 1 shows, the React client communicates with the Express API over HTTPS, and the API centralizes all business logic, including authentication and role based access control, listing management, the matching engine, the claim verification workflow, and the notification service. The API is the only component that talks directly to MongoDB, the email provider, and the map tiles service, which keeps the front end simple and keeps sensitive operations, such as database writes and outgoing email, on the server side.

The claim verification workflow, one of the more involved pieces of business logic in the system, is illustrated in more detail in Figure 2. It begins once the matching engine surfaces a potential match and ends either with the listing being marked resolved or with the claim being rejected and the listing staying active for other potential claimants.

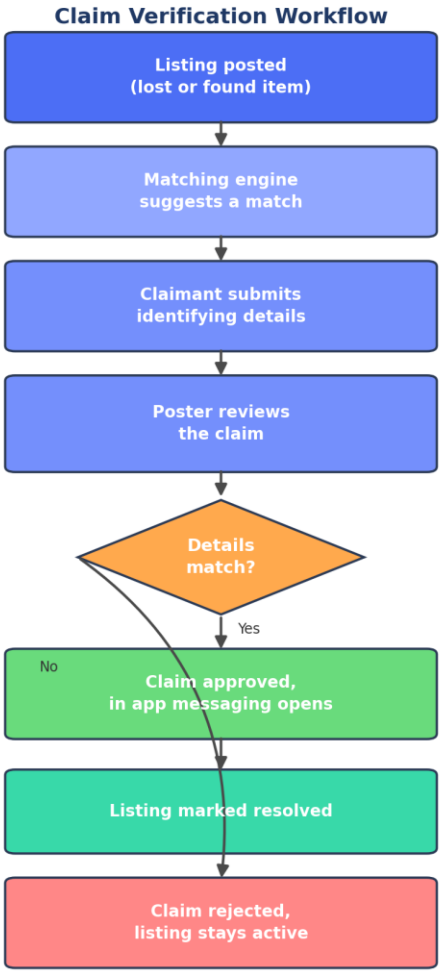


Figure 2. Claim verification workflow, from an initial match suggestion to a resolved or rejected claim.

3. Project Timeline

The timeline below follows the official deliverable schedule posted on Canvas for CSCE 5430. The semester is organized around five graded project deliverables, each due at 11:59pm on the date shown. The team uses the shared Trello Kanban board, with columns for To Do, Doing, Stalled, and Done, to track weekly progress and each member's status against this plan between deliverable dates.

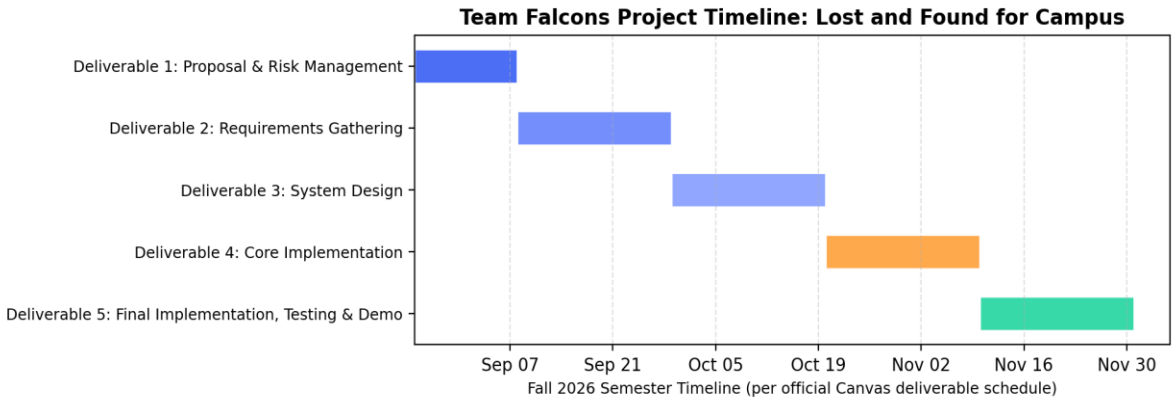


Figure 3. Gantt chart showing the five graded project deliverables across the Fall 2026 semester, per the Canvas schedule.

The table below expands on each deliverable, describing its purpose and expected output. Deliverable 1 additionally requires a PowerPoint submission and a five minute presentation video, both due the same day as the written report, and an individual peer evaluation due the following day.

Deliverable	Due Date	Purpose and Expected Output
Deliverable 1	Sep 7, 11:59pm	Project proposal, project plan, and risk management report, together with a PowerPoint submission and a five minute presentation video. An individual peer evaluation for Deliverable 1 is due separately on Sep 8, 11:59pm.
Deliverable 2	Sep 28, 11:59pm	Gather and document detailed functional and non functional requirements and user stories for each module described in the project description.
Deliverable 3	Oct 19, 11:59pm	Design the system: UI and UX wireframes, the MongoDB data schema, and the REST API endpoints that the front end will consume.
Deliverable 4	Nov 9, 11:59pm	Implement the core application: CRUD listings, authentication, search and filter, and begin the matching engine, notification service, and claim verification workflow.
Deliverable 5	Nov 30, 11:59pm	Complete remaining features (map view, in app messaging, admin and analytics dashboard, role based access control), finish testing, and submit the final report, PowerPoint, and a nine to ten minute final project presentation and demo video.

4. Risk Management

The team identified the following top three risks for this project. Each risk includes why it matters to the project, how the team will monitor it, how the team will reevaluate it as the semester progresses, and a contingency plan if the risk materializes.

Risk 1: Uneven Team Contribution

Why it matters: With four team members and a shared grade, an uneven split of work can quickly become unfair and can also put the deliverable itself at risk if a module owner falls behind without the rest of the team noticing in time.

- Monitoring: Weekly review of each member's Trello cards and GitHub commit activity, along with brief WhatsApp check ins between our Tuesday and Thursday meetings.

- Reevaluation: Contribution balance is reassessed at every deliverable milestone, using the member contribution table required in each submission as a checkpoint.
- Contingency: If a member's contribution consistently falls short, the team leader will have a direct conversation first, following the same approach the team already agreed on in our team planning document. If the issue persists, the team will document it and escalate to the instructor or TA per course policy.

Risk 2: Scope Creep on the Expanded Feature Set

Why it matters: The expanded feature set, including the matching engine, notifications, claim verification, messaging, the map view, and the admin dashboard, is significantly more ambitious than a basic CRUD application. Without active management, the team could run out of time before finishing the features already promised to the instructor.

- Monitoring: Progress toward each deliverable is tracked on the Trello board against the milestones shown in the Gantt chart, and scope is reviewed at the start and end of work on each deliverable.
- Reevaluation: At the end of work toward each deliverable, the team compares actual progress to plan and decides whether upcoming features need to be simplified or rescheduled.
- Contingency: The team maintains a core versus stretch feature split. Core features are the listings, search, and resolve workflow; stretch features are the matching engine, messaging, the map view, and admin analytics. Stretch features are simplified or cut first if the team falls behind schedule.

Risk 3: Third Party Integration and Tooling Risk

Why it matters: The project depends on several external services, including a map tiles provider, an email provider, and hosted database and application platforms. Any of these could introduce setup delays, unexpected configuration issues, or free tier usage limits that block progress if they are not addressed early.

- Monitoring: Map API, email service, and database hosting integrations are set up and smoke tested early, in isolated branches, rather than left until final integration.
- Reevaluation: If a chosen service proves unreliable or approaches its free tier limits while working on Deliverable 4, the team revisits that choice before it can block Deliverable 5.
- Contingency: Free and open fallbacks are identified in advance. Leaflet with OpenStreetMap requires no API key and is already the default map choice. Mailtrap or Ethereal can be used for email testing if a production email provider is delayed, and MongoDB Atlas's free tier remains the default database host throughout development.

5. Team Roles

Team roles were assigned based on the preferences each member identified in our team planning document, matched against the modules described in the project description. Figure 4 summarizes the reporting structure, and the paragraphs below describe each member's responsibilities in more detail.

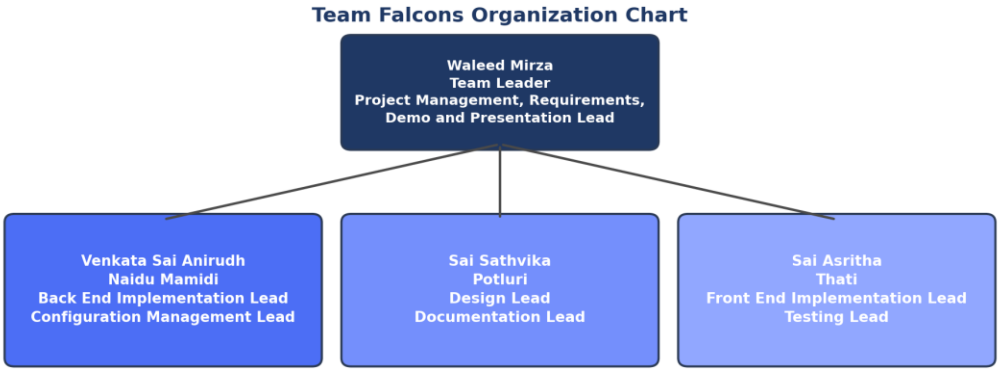


Figure 4. Team Falcons organization chart, showing the team leader and each member's primary leadership roles.

Waleed Mirza serves as Project Management Lead, Requirements Lead, and Demo and Presentation Lead. In this role he coordinates the team schedule and deliverable submissions, communicates with the instructor and TA on the team's behalf, leads the requirements gathering process for Deliverable 2, owns the design and implementation of the claim verification workflow described in Figure 2, and leads the final demo and presentation at the end of the semester.

Venkata Sai Anirudh Naidu Mamidi serves as Back End Implementation Lead and Configuration Management Lead. He owns the Node.js and Express API, the MongoDB database schema, and the matching engine and notification service described in the project description. As Configuration Management Lead, he also manages the structure of the GitHub repository and the team's branching and merge process.

Sai Sathvika Potluri serves as Design Lead and Documentation Lead. She owns the UI and UX design for the application, including wireframes for the campus map view and the admin dashboard, and leads documentation efforts across the project, including requirements documentation, design decisions, and meeting minutes.

Sai Asritha Thati serves as Front End Implementation Lead and Testing Lead. She owns the React front end, including the listings interface and the in app messaging interface, and leads unit and integration testing across the application as features are completed for each deliverable.

6. Member Contribution Table

Work on this deliverable was split fairly evenly across all four team members, as detailed below.

Member Name	Contribution Description	Overall Contribution (%)
Waleed Mirza	Compiled and formatted the Deliverable 1 report, wrote the project description and the claim verification workflow detail, drafted the project timeline, and set up the GitHub repository and Trello board.	25%
Venkata Sai Anirudh Naidu Mamidi	Wrote the development environment and technology stack details, wrote the third party integration risk section, and reviewed the repository directory structure.	25%

Sai Sathvika Potluri	Wrote the scope creep risk section, contributed to the project description and stakeholder discussion, and drafted the initial meeting minutes.	25%
Sai Asritha Thati	Wrote the Team Roles section and the uneven contribution risk section, compiled the member contribution table, and proofread the final report.	25%

Note: Items 3 (README with directory structure), 4 (repository directory structure), and 5 (initial meeting minutes) required by the Deliverable I instructions are submitted directly in the team's GitHub repository, as specified in the assignment.